

Eco-Tech Kampanya Motoru

Eco-Tech Kampanya Motoru — Proje Raporu

1. Problem Tanımı
2. Kullanılan Model ve Mimari
3. En Önemli Promptlar ve Tasarım Gerekçeleri
4. Karşılaşılan Zorluklar
5. Etik Değerlendirme

Prompt Log Raporu

- TUR 1 - Proje baslangic promptu
- TUR 2 - Ilerleme onayi + adim birlestirme
- TUR 3 - Etik ilkeli endpoint tasarimi promptu
- TUR 4 - Meta-prompt (Gemini promptu yazdirma) + controller kodu
- TUR 5 - Ilerleme onayi
- TUR 6 - Hata ayiklama / dayaniklilik promptu
- TUR 7 - Dokumantasyon (DX) promptu
- TUR 8 - Gereksinim duzeltme (iterasyon)
- TUR 9 - Raporlama promptu
- TUR 10 - Frontend (UI/UX) tasarim promptu

OZET TABLO

SON NOT

Etik Degerlendirme

SAMSUNG x UNDP · GENAI HACKATHON

Eco-Tech Kampanya Motoru

Hackathon Teslim Dosyasi — Proje Raporu, Prompt Log ve Etik Degerlendirme

Eco-Tech Kampanya Motoru — Proje Raporu

Samsung x UNDP GenAI Hackathon Teslimi

1. Problem Tanımı

Sürdürülebilirlik kampanyalarında aynı mesajın farklı hedef kitlelere (genç tüketici, kurumsal partner, mevcut müşteri) uyarlanması zaman alan ve tutarlılığı zor bir pazarlama işidir; üstelik yapay zekâ ile üretilen çevre iddialarının doğrulanmadan yayılması “greenwashing” riski taşır. Proje, Samsung x UNDP ortak geri dönüşüm kampanyası senaryosu üzerinden bu problemi çözer: kullanıcı tek cümlelik bir kampanya fikri girer; sistem bu fikri Z Kuşağı için Instagram postu, kurumsal partnerler için LinkedIn e-postası ve mevcut müşteriler için SMS olmak üzere üç formata dönüştürür. Ayırt edici iddiamız, içerik üretiminin denetlenebilir ve insan onaylı bir süreç olarak tasarlanmasıdır: hiçbir çıktı otomatik yayınlanmaz, her üretim kalıcı olarak izlenebilir.

2. Kullanılan Model ve Mimari

İçerik üretimi için Google Gemini API (gemini-2.5-flash, Google AI Studio ücretsiz katmanı) kullanıldı; entegrasyon Spring AI’nin Google GenAI starter’ı üzerinden, Vertex AI’a ve GCP faturalandırmasına ihtiyaç duymadan yapıldı. Backend, Java 21 / Spring Boot ile Controller → Service → Gemini → PostgreSQL akışında çalışır: istek doğrulanır, versiyonlanmış prompt şablonuyla Gemini’ye gönderilir, dönen yanıt katı JSON şemasına karşı doğrulanır ve üretim parametreleriyle birlikte veritabanına kaydedilir. Tüm sistem (uygulama + PostgreSQL) Docker Compose ile tek komutta ayağa kalkar; API key ve veritabanı bilgileri yalnızca ortam değişkenlerinden okunur.

Frontend tarafında **React** kullanıldı: kullanıcının kampanya fikrini girdiği tek sayfalık arayüz, backend’in /api/campaigns endpoint’ine REST üzerinden istek atar ve dönen instagramPost / linkedinEmail / sms alanlarını üç ayrı sonuç kartında gösterir. Arayüz backend’den bağımsız bir istemci olarak tasarlandı; bu sayede CORS konfigürasyonu üzerinden farklı origin’den (örn. localhost:5173) sorunsuz iletişim kurulabildi.

3. En Önemli Promptlar ve Tasarım Gerekçeleri

Gemini içerik üretim promptu (PromptTemplateV1). Altı elementli yapıyla (Role / Context / Goal / Constraints / Style / Output Format) tasarlandı ve zero-shot olarak kullanıldı: modele örnek çıktı verilmedi; kalite, kimlik ataması ve format başına ayrıntılı kısıtlarla (SMS 160 karakter, Instagram 80-150 kelime + hashtag, LinkedIn 120-200 kelime) sağlandı. Chain-of-thought tekniği “gizli” biçimde uygulandı: modelden yanıt yazmadan önce her kitle için ton, ana mesaj ve eylem çağrısını adım adım planlaması, ancak bu düşünme sürecini çıktıya yazmaması istendi — böylece planlamanın kalite katkısı korunurken JSON çıktısını bozacak serbest metin riski ortadan kalktı. Few-shot örnekleme bu projede kullanılmadı. En kritik kısıt halüsinasyona karşıdır: kullanıcı girdisinde veya doğrulanmış veri listesinde bulunmayan hiçbir sayısal iddianın üretilmemesi, gereken yerde bunun yerine “[KAYNAK DOĞRULANMALI]” etiketinin yazılması zorunlu kılındı. Prompt, “promptTemplateVersion” alanıyla versiyonlanır; hangi kaydın hangi şablon sürümüyle üretildiği her zaman izlenebilir.

Etik ilkeli endpoint tasarım promptu. Geliştirme asistanına verilen bu prompt, FATS ilkelerinin (Fairness, Accountability, Transparency, Safety) API davranışına çevrilmesini şart koştu: etik, prompt süslemesi değil mimari gereksinim olarak tanımlandı. Gerekçe: jüri kriterlerindeki “etik farkındalığın” kodda doğrulanabilir karşılıklar (durum makinesi, audit alanları, uyarı mekanizması) üretmesiydi. Sonuçta beş endpoint’lik tasarım, DRAFT → APPROVED/REJECTED insan onay akışı ve her yanıtta dönen şeffaflık metadata’sı bu prompttan doğdu.

Hata ayıklama ve dayanıklılık promptu. Gemini 503 hatası üzerine yazılan bu prompt, kök neden teşhisi, noktasal değişiklik ve aşırı mühendislik yasağı kısıtlarıyla yapılandırıldı; gerekçe, hackathon temposunda çözümün mevcut mimariyi bozmadan uygulanabilir kalmasıydı.

4. Karşılaşılan Zorluklar

Geliştirme sırasında yaşanan somut sorunlar: (1) Gemini’nin “high demand” kaynaklı geçici 503 hataları — exponential backoff’lu yeniden deneme (4 deneme, 2s→4s→8s) eklendi; denemeler tükenirse istemciye Retry-After başlığıyla 503 dönülür ve başarısız deneme de denetim izine kaydedilir. Bu noktada 502/503 ayrımı tartışıldı ve durumun geçiciliğini doğru ifade ettiği için 503 seçildi. (2) Spring AI’nin Google GenAI modülünde konfigürasyona project-id/location eklenirse istemcinin sessizce Vertex AI moduna geçip ücretsiz API key’i reddetmesi — yalnızca api-key uyarı kullanılarak önlendi. (3) Modelin, yasaklanmasına rağmen yanıtı ara sıra markdown code-fence içine sarabilmesi — parse öncesi savunmacı temizlik katmanı eklendi. (4) Başarısız üretimlerin de kaydedilmesi hedefi ile transaction yönetiminin çelişmesi: üretim metodu transaction kapsamına alınsaydı, fırlatılan hata az önce yazılan RED kaydını geri alacaktı; kayıt bütünlüğü için üretim akışı bilinçli olarak transaction dışında tutuldu. (5) Kurulumun yalnızca Docker gerektirmesi istendiğinde derlemenin de konteynere taşınması gerekti; çok aşamalı (multi-stage) Docker imajıyla çözüldü.

5. Etik Değerlendirme

Riskler ve önlemler. (1) Uydurma istatistik/rakam üretimi: iki katmanlı önlem alındı — prompt seviyesinde sayısal iddia

yaşığı ve “[KAYNAK DOĞRULANMALI]” etiketi zorunluluğu; backend seviyesinde modelden bağımsız bir tarama, girdide bulunmayan sayısal iddiaları tespit edip kayda CONTAINS_UNVERIFIED_STATISTIC uyarısı ekler. İlke: modele güvenme, doğrula. (2) AI çıktısının insan onayı olmadan yayınlanması: her içerik DRAFT statüsünde doğar; yalnızca ayrı bir onay isteğiyle APPROVED/REJECTED olabilir, geçiş kuralları sunucuda zorlanır, karar ve zamanı kaydedilir (human-in-the-loop). (3) Şeffaflık eksikliği: her yanıt, içeriğin AI ürünü olduğunu, model adını, temperature değerini ve prompt sürümünü içeren metadata taşır; hatalar da tutarlı ve açıklayıcı formatta döner. (4) Hesap verebilirlik: her kayıt, Gemini’ye giden promptun tam metni dahil üretim bağlamını taşır; doğrulamadan geçemeyen veya sağlayıcıya ulaşamayan başarısız denemeler bile otomatik RED notuyla kalıcı olarak loglanır. (5) Kitle stereotipleştirme: prompt, kitle segmentasyonu gereği genellemeler içerir; bu risk otomatik çözülmedi, ana savunma insan onay katmanıdır — klişe veya dışlayıcı ton onay aşamasında reddedilebilir.

Bilinen sınırlılıklar. Kimlik doğrulama kapsam dışı bırakıldığı için onayı “kimin” verdiği tutulamıyor; istatistik taraması regex tabanlı ve kabadır (yazıyla yazılmış sayıları kaçırabilir); modelin prompt kısıtlarına uyması garanti değildir — backend taraması bu yüzden vardır ama o da eksiksiz değildir; önyargı ve toksisite için otomatik sınıflandırıcı kullanılmadı, bu denetim bilinçli olarak insana bırakıldı ve burada açıkça belgelendi. -e

Prompt Log Raporu

PROMPT LOG RAPORU

Proje: Eco-Tech Kampanya Motoru (Samsung x UNDP GenAI Hackathon)
Arac: Claude (gelistirme asistani) + Google Gemini (uretim modeli)
Not: Tur numaralari kullanicinin gonderdigi promptlari izler.

TUR 1 - Proje baslangic promptu

PROMPT:

ROL

Sen Spring Boot, PostgreSQL, Docker ve Google Gemini API entegrasyonu konusunda uzman, kideimli bir backend muhendisisin. Bana bu projeyi sifirdan, adim adim insa etmemde rehberlik edeceksin.

PROJE FIKRI VE SENARYO

"Eco-Tech Kampanya Motoru" adli bir hackathon projesi gelistiriyorum. Senaryo: Samsung ve UNDP ortak bir geri donusum/surdurulebilirlik kampanyasi yurutuyor. Kullanici arac tek cumlelik basit bir kampanya fikri giriyor (ornek: "Eski telefonunu getir, geri donusturelim, karbon ayak izini azaltalim."). Sistem bu fikri Gemini API'ye gonderip, uc farkli hedef kitleye uyarlanmis icerik uretiyor:

- Z Kusagi icin Instagram Postu (dinamik, emoji, "Iklim Eylemi" ve "Gelecegini Korum" vurgulu)
- Kurumsal Partnerler icin LinkedIn E-postasi (Samsung'un karbon notr hedefleri ve UNDP dongusel ekonomi verileriyle desteklenmis, ciddi ton)
- Mevcut Musteriler icin SMS (dogrudan eyleme cagri)

Uretilen her kampanya, denetlenebilirlik (audit trail) ve sorumlu AI kullanimi ilkesi geregi veritabanina kaydedilecek; kullanici gecmis kampanyalari listeleyebilecek.

TEKNİK STACK

- Backend: Spring Boot (Java, Maven)
- Veritabanı: PostgreSQL (Docker ile ayaga kadirilacak)
- AI Model: Google Gemini API (Google AI Studio uzerinden ucretsiz API key ile, Vertex AI DEĞİL, GCP faturalandirma hesabi istemiyorum)
- Framework entegrasyonu: Spring AI

İSTEDİĞİM İLERLEME SİRASI

Beni sirayla su adimlardan gecir, her adimda onceki adimin tamamladigini dogrulamadan bir sonrakine gecme:

- Gemini API Key Alma
- Docker + PostgreSQL Kurulumu
- Spring Boot Proje Iskeleti ve Bagimliliklar
- Entity ve Repository Tasarimi
- Endpoint Tasarimi
- Servis Katmani ve Gemini Entegrasyonu
- Hata Yonetimi

KISITLAR

- Basit ve hackathon temposuna uygun bir mimari kur; production-grade karmaşiklik (auth, rate limiting, microservice ayrimi vb.) ekleme.
- API key ve DB credential'lari asla hardcoded etme, ortam degiskenlerinden oku.
- Kod calisir durumda, yorum satirlariyla aciklanmis olsun.
- Her adimda bana ne yapmam gerektigini net sekilde soyle, sonraki adima ben onay vermeden gecme.

PROMPT ÖZETİ:

Rol ataması (kademli backend mühendisi: Spring Boot, PostgreSQL, Docker, Gemini API) + proje senaryosu + teknik stack + 7 adımlık ilerleme sırası + kısıtlar (basit mimari, credential hardcoded yasagi, onaysız adım atlamama).

KULLANILAN TEKNİK(LER):

- Role prompting
- Görev ayırma (task decomposition - 7 adım)
- Kısıt belirleme (constraints)
- Kapalı ilerleme (her adımda kullanıcı onayı şartı)
- Çıktı sırası tanımlama

TASARIM GEREKÇESİ:

Tüm projeyi tek seferde istemek yerine, doğrulanabilir küçük adımlarla ilerleyen bir geliştirme oturumu kurmak için tasarlandı. Böylece her katman bir önceki sağlam temele oturuyor ve hata ayıklama kolaylaşıyor.

SONUC:

Adım 1 (Gemini API key alma) güncel bilgiyle, güvenli saklama önerileriyle tamamlandı.

TUR 2 - İlerleme onayı + adım birleştirme

PROMPT (özet - doğal dil):

Kısa onay + Adım 2 (Docker+PostgreSQL) ile Adım 3'un (Spring bağımlılıkları) birlikte işlenmesi talebi.

KULLANILAN TEKNİK(LER):

- İteratif yönlendirme (conversational steering)
- Kapsam birleştirme

TASARIM GEREKÇESİ:

Hackathon zaman kısıtı nedeniyle iki kurulum adımını tek turda tamamlamak, gereksiz tur kaybını önlemek için tercih edildi.

SONUC:

docker-compose.yml, .env, pom.xml (Spring AI BOM + google-genai starter) ve application.yml üretildi; Vertex AI moduna düşme tuzağı tespit edildi ve onldi.

TUR 3 - Etik ilkel endpoint tasarımı promptu

PROMPT:

ROL

Sen Spring Boot REST API tasarımında uzman, aynı zamanda Sorumlu AI (Responsible AI) ilkelerini backend mimarisine gömmе konusunda deneyimli bir kademli mühendissin.

BAGLAM

"Eco-Tech Kampanya Motoru" projesinin backend'i (Spring Boot + PostgreSQL + Gemini API) ayakta. Şimdi REST endpoint'lerini tasarlıyoruz. Bu proje bir hackathon değerlendirmesinde "etik farkındalık" kriteriyle puanlanacak, bu yüzden endpoint tasarımının kendisi de etik ilkeleri (Fairness, Accountability, Transparency, Safety - FATS ilkesi) somut şekilde yansıtmalı; etik yalnızca prompt seviyesinde değil, API davranışında da görünür olmalı.

Sistem: Kullanıcı kampanya fikri gönderiyor -> Gemini içerik üretiyor -> sonuç veritabanına kaydediliyor -> kullanıcı geçmiş kampanyaları görüntüleyebiliyor.

GOREV

Bana aşağıdaki prensipleri karşılayan eksiksiz bir endpoint tasarımı ver:

1. Accountability (Hesap Verebilirlik): Her üretilen içerik kalıcı olarak loglanmalı - hangi promptla, hangi girdiyle, ne zaman üretildiği izlenebilir olmalı (audit trail).
2. Transparency (Şeffaflık): Kullanıcı, bir kampanya çıktısının "AI tarafından üretildiğini" ve hangi model/parametrelerle üretildiğini görebilmeli.
3. Human-in-the-loop (İnsan Denetimi): Üretilen içerik otomatik yayınlanmıyor; bir insanın onaylaması/reddetmesi gerekiyor. Status mekanizması (draft/approved/rejected) öner.
4. Safety (Güvenlik/Hata Toleransı): Gemini API'den hatalı, eksik veya potansiyel olarak yanıltıcı (uydurma istatistik içeren) çıktı gelirse bunu nasıl yakalayip kullanıcıya/loga yansıtacağımı belirt.
5. Bu dört ilkeyi karşılayan TAM endpoint listesini tablo halinde ver.
6. Her endpoint için, hangi FATS ilkesine hizmet ettiğini kısa bir notla belirt.

KISITLAR

- Auth/login sistemi kurmuyoruz, kapsam dışı bırak.
- Asiri karmaşık bir onay iş akışı istemiyorum - tek seviye basit bir durum alanı yeterli.
- Endpoint sayısını gereksiz cogaltma; CRUD + generate + durum güncelleme mantıklı bir minimumda kalsın.

ÇIKTI FORMATI

1. Endpoint tablosu (metod, path, request, response, hata durumları)
2. Her endpoint için 1 cümlelik "hangi etik ilkeye hizmet ediyor" notu
3. Response semalarında seffaflik/hesap verebilirlik için önerilen ek alanların listesi

PROMPT ÖZETİ:

Rol ataması (REST API + Responsible AI uzmanı) + bağlam (FATS ilkeleri: Fairness, Accountability, Transparency, Safety) + görev (5 gereksinim) + kısıtlar (auth yok, minimum endpoint) + çıktı formatı (tablo + ilke gerekçesi + alan listesi).

KULLANILAN TEKNİK(LER):

- Role prompting
- Deger/ilke enjeksiyonu (etik gereksinimlerin teknik spesifikasyona çevrilmesi)
- Çıktı formatı dayatma
- Negatif kısıtlar (yapılmayacaklar listesi)

TASARIM GEREKÇESİ:

Etigi prompt süslemesi olarak değil, API davranışı (status makinesi, audit alanları, safety uyarıları) olarak tasarlamak için kuruldu. Böylece juriye "etik prensipler prompt'ta değil, mimaride" denilebilir hale geldi.

SONUC:

5 endpoint'lik tasarım; DRAFT/APPROVED/REJECTED durum makinesi; aiMetadata ve safety.warnings alanları; iki seviyeli doğrulama kararı (yapısal hard-fail + içerik soft-warning).

TUR 4 - Meta-prompt (Gemini promptu yazdırma) + controller kodu

PROMPT ÖZETİ:

İki isli prompt.

Is 1: Gemini'ye gidecek sistem promptunun 6 elementli yapıyla (Role/Context/Goal/Constraints/Style/Output Format) yazdırılması; uydurma istatistik yasası + "[KAYNAK DOĞRULANMALI]" etiketi; chain-of-thought'un "uygulanıp çıktıya yazılmaması"; versiyonlanabilirlik. Is 2: 5 endpoint'in Swagger, validasyon ve doğru HTTP statuslarla implementasyonu.

KULLANILAN TEKNİK(LER):

- Meta-prompting (prompt üreten prompt)
- 6 elementli master şablon (Role/Context/Goal/Constraints/Style/Output Format)
- Gizli zincirleme düşünme (hidden Chain-of-Thought)
- Halüsinasyon kısıtı (anti-hallucination constraint)
- Çıktı seması dayatma (kati JSON)
- Prompt versiyonlama (promptTemplateVersion)

TASARIM GEREKÇESİ:

Üretim modelinin (Gemini) çıktısını hem kaliteli hem makine tarafından doğrulanabilir kılmak; denetlenebilirliği prompt seviyesine indirmek için tasarlandı. CoT adımı bilinçli olarak "çıktıya yazma" şeklinde kısıtlandı ki JSON parse işlemi bozulmasın.

SONUC:

PromptTemplateV1 metni + element-satır eşleştirmesi; controller, DTO'lar, GlobalExceptionHandler kodları üretildi.

TUR 5 - İlerleme onayı

PROMPT (özet - doğal dil):

"evet geçebilirsiniz" - kısa onay.

KULLANILAN TEKNİK(LER):

- Kapalı ilerleme onayı

TASARIM GEREKÇESİ:

Önceki adımların (endpoint tasarımı, Gemini promptu) doğrulanmasından sonra entity/repository/service implementasyon adımı bilinçli geçiş.

SONUC:

Campaign entity, repository, PromptTemplateV1 sınıfı, CampaignServiceImpl (Gemini çağırışı, JSON parse, regex istatistik taraması, audit kaydı) üretildi; başarısız üretimlerin de kaydedilmesi için bilinçli @Transactional'siz tasarım kararı alındı.

TUR 6 - Hata ayıklama / dayanıklılık promptu

PROMPT:

```
# ROL
Sen Spring AI, resilience (dayaniklilik) mühendisliği ve hata yönetimi
konusunda uzman bir kıdemli backend mühendisisin.

# BAGLAM
"Eco-Tech Kampanya Motoru" projesinde CampaignServiceImpl.generate()
metodu içinde Gemini API'ye ChatClient üzerinden istek atıyorum.
Aşağıdaki hatayı aldım:
[Log/stack trace]
Kok neden: com.google.genai.errors.ServerException: 503 - "This model
is currently experiencing high demand." Spring AI'in kendi retry
mekanizması (SpringAiRetryAutoConfiguration) 1 kez denemis ve yine
başarısız olmuş, sonuç olarak RuntimeException fırlatılmış.

# GOREV
1. Bu hatanın kok nedenini teşhis et: kod hatası mı, yoksa Gemini
API'nin gecici (transient) bir kapasite sorunu mu, netleştire.
2. Spring AI'in default retry davranışını açıkla ve application.yml
üzerinden daha dayanıklı hale getir.
3. Retry'lar tükendiğinde bu hatayı GlobalExceptionHandler'da nasıl
yakalayıp anlamlı bir response'a çevireceğimi göster (doğru HTTP
status, kullanıcıya anlamlı mesaj, başarısız denemenin loglanması).
4. Demo sırasında tekrar yaşanmaması için pratik önlem öner.

# KISITLAR
- Kodumu bastan yazma, sadece ilgili sınıflarda NOKTASAL değişiklikleri
göster.
- Asiri mühendislik yapma (Resilience4j gibi ağır bir kütüphane
eklemek yerine Spring AI'in retry mekanizmasını doğru konfigure
etmeyi önceliklendir).

# CIKTI FORMATI
1. Kok neden teşhisi (kısaca)
2. application.yml'de retry/backoff konfigürasyonu
3. GlobalExceptionHandler'a eklenecek kısım
4. Demo riskini azaltmak için ek öneri

---

PROMPT ÖZETİ:
Rol ataması (resilience uzmanı) + hata bağlamı (Gemini 503
ServerException, retry'in yetersiz kalması) + 4 maddelik görev (kok
neden, retry konfigürasyonu, handler güncellemesi, demo önemi) +
kısıtlar (noktasal değişiklik, asiri mühendislik yasası).

KULLANILAN TEKNİK(LER):
- Hata bağlamı enjeksiyonu (error-context injection)
- Role prompting
- Kapsam sınırlama (noktasal değişiklik kısıtı)
- Gerektelendirme talebi (502 vs 503 karşılaştırması)

TASARIM GEREKÇESİ:
Gecici provider hatalarına karşı sistemi dayanıklı kılmak ve hatayı
kullanıcıya şeffaf yansıtmak için tasarlandı; asiri mühendislikten
kaçınılarak hackathon temposuna uygun, mevcut Spring AI altyapısı
üzerinden çözüm arandı.

SONUÇ:
spring.ai.retry konfigürasyonu + programatik RetryTemplate; 502 yerine
503 + Retry-After kararı; başarısız denemenin de REJECTED olarak audit
kaydına geçmesi kararı; demo için hazır yedek veri taktiği.
```

TUR 7 - Dokümantasyon (DX) promptu

PROMPT:

```
# ROL
Sen geliştirici deneyimi (DX) ve proje dokümantasyonu konusunda uzman
bir kıdemli backend mühendisisin.

# BAGLAM
"Eco-Tech Kampanya Motoru" projemi (Spring Boot + PostgreSQL/Docker +
Gemini API) tamamladım, GitHub'a push edeceğim. Takım arkadaşım repoyu
klonlayıp projeyi kendi bilgisayarında ayaga kaldıracak. Arkadaşımın
teknik seviyesi projeyi çalıştırmaya yeter ama karmaşık kurulum
adımlarıyla vakit kaybetmemeli.

# GOREV
1. Ön Gereksinimler (minimum liste)
2. Ortam Degiskenleri (Gemini API key nereye/nasıl girilecek,
.env.example şablonu)
3. Tek Komutla Ayaga Kaldırma
4. Doğrulama (Swagger UI, health check, örnek istek)
5. Sık Karşılansılalabilecek 2-3 Basit Hata ve Çözümü

Bunu bir README.md dosyası olarak, başlıklar halinde hazırla.

# KISITLAR
- Karmaşık alternatif kurulum yöntemlerinden bahsetme - tek net bir
yol göster.
- API key/credential girilecek her yeri tek yerde topla.
- Adım sayısını mümkün olduğunca azalt.

# CIKTI FORMATI
```

Dogrudan kopyalanabilir, tam bir README.md icerigi.

PROMPT OZETI:

Rol atamasi (DX/dokumantasyon uzmani) + hedef kitle tanimi (teknik ama zamani kisitli takim arkadasi) + 5 maddelik icerik gereksinimi + kisitlar (tek kurulum yolu, opsiyonel dallanma yasagi, credential'larin tek yerde toplanmasi).

KULLANILAN TEKNİK(LER):

- Hedef kitleye gore yazim (audience-aware prompting)
- Persona tanimi
- Negatif kisitlar
- Cikti formati dayatma (README.md)

TASARIM GEREKCESI:

Repoyu klonlayan kisinin minimum adimda projeyi calistirmasi hedeflendi; hackathon gunu zaman kisitli oldugu icin karmasiklik bilincli olarak elendi.

SONUC:

README.md (.env sablonu, kisa komutlu kurulum, dogrulama adimlari, sik hatalar bolumu) uretildi.

TUR 8 - Gereksinim duzeltme (iterasyon)

PROMPT (ozet - dogal dil):

"Docker'la kaldirsin, Java kurmasin" - kisa dogal dil gereksinim degisikligi.

KULLANILAN TEKNİK(LER):

- Iteratif gereksinim rafine etme (requirement refinement)
- Onceki ciktinin revizyonu

TASARIM GEREKCESI:

On gereksinimi yalnızca Docker'a indirmek; arkadasin makinesinde JDK kurulu olmasa bile projeyi calistirabilmesini saglamak icin.

SONUC:

Multi-stage Dockerfile + docker-compose'a app servisi eklendi; README'nin Docker-only akisa gore yeniden yazilmasi saglandi.

TUR 9 - Raporlama promptu

PROMPT OZETI:

Session'in kendisinin kaynak olarak kullanilmasi: prompt log raporu + ozet tablo + etik degerlendirme uretilmesi talebi; "gercekte gecmeyi uydurma" kisiti; duz metin (.txt) formati talebi.

KULLANILAN TEKNİK(LER):

- Oz-referansli ozetleme (session'i veri olarak kullanma)
- Durustluk kisiti
- Format dayatma (.txt, markdown suslemesiz)

TASARIM GEREKCESI:

Hackathon teslimi icin izlenebilir bir prompt gecmisi ve etik degerlendirme metni uretmek; raporun juri tarafından dogrudan okunabilir/kopyalanabilir olmasi hedeflendi.

SONUC:

Bu PROMPT_LOG.txt dosyasi ve ayri bir ETIK_DEGERLENDIRME.txt dosyasi.

TUR 10 - Frontend (UI/UX) tasarim promptu

PROMPT:

Kendini Senior UI/UX Designer ve Frontend Developer olarak dusun. "Eco-Tech Campaign Engine" isimli, yapay zeka destekli surdurulebilir kampanya uretim araci icin modern ve premium bir web arayuzu tasarla.

Tasarim:

- OpenAI, Microsoft Copilot, Vercel, Stripe ve Notion AI seviyesinde profesyonel gorunum
- Mavi, lacivert, beyaz ve cyan renk paleti
- Glassmorphism, gradient arka planlar ve modern kart tasarımlari
- Yumusak golgeler, hover efektleri ve akici animasyonlar
- Tam responsive (masaustu, tablet ve mobil)

Sayfada su bolumler bulunsun:

- Ust menu (Logo + Menu + Basla butonu)
- Hero alani (Baslik, aciklama, kampanya fikri girisi ve "Kampanyayi Olustur" butonu)

- Sonuclarin gosterilecegi 3 kart:
 1. Instagram Gonderisi
 2. E-posta Kampanyasi
 3. Surdurulebilirlik Etkisi
- Ozellikler bolumu
- Modern footer

Kod tek bir HTML dosyasinda olsun. CSS ve JavaScript dosyanin icinde yer alsin. Kod temiz, okunabilir ve profesyonel olsun. Amac okul odevi gorunumunde degil, gercek bir AI SaaS urununun arayuzune benzeyen premium bir tasarim elde etmek.

PROMPT OZETI:

Rol atamasi (Senior UI/UX Designer + Frontend Developer) + referans urun seviyesi tanimi (OpenAI, Copilot, Vercel, Stripe, Notion AI) + gorsel dil kisitlari (renk paleti, glassmorphism, gradient, animasyon) + zorunlu sayfa bolumleri (header, hero, 3 sonuc karti, ozellikler, footer) + teknik format kisiti (tek HTML dosyasi, gomulu CSS/JS).

KULLANILAN TEKNIK(LER):

- Role prompting (Senior UI/UX Designer + Frontend Developer)
- Referans ornekli stil tanimlama (reference-based style anchoring - rakip/benzer urunler uzerinden kalite beklentisi belirleme)
- Negatif kisit ("okul odevi gorunumunde degil" - istenmeyen sonucu acikca disleyerek beklenen kaliteyi netlestirme)
- Yapisal ayristirma (sayfanin bolum bolum tanimlanmasi: header, hero, 3 sonuc karti, ozellikler, footer)
- Cikti formati dayatma (tek dosya, gomulu CSS/JS kisiti)

TASARIM GEREKCESI:

Bu prompt, arka planda gelistirdigimiz backend'in (Gemini uzerinden uretilen Instagram/LinkedIn/SMS icerikleri) kullanicu tarafinda nasil sunulacagini tanımlamak icin yazildi. Referans urun isimleri (OpenAI, Stripe, Vercel vb.) bilincli olarak somut bir kalite cipasi olarak kullanildi; Boylece modelin "guzel bir arayuz" gibi belirsiz bir talimati kendi basina yorumlamasi yerine, net ve dogrulanabilir bir hedefe gore uretim yapmasi saglandi. Sonuc ciktisinin (3 kart: Instagram/E-posta/Surdurulebilirlik Etkisi) backend'in ChatModel yanitindaki instagramPost/linkedinEmail/sms alanlarina birebir karsilik gelecek sekilde tasarlanmasi, prompt tasarim asamasinda bilincli olarak saglandi.

SONUC:

Tek dosyalik, glassmorphism ve gradient temali, responsive bir HTML/CSS/JS arayuz taslagi; backend'in ChatModel/CampaignService cikti semasiyla dogrudan eslesen 3 sonuc karti duzeni.

OZET TABLO

Tur	Kullanilan teknik(ler)	Kisa amac
1	Role prompting, gorev ayristirma, kapili ilerleme, kisit belirleme	Oturumu adım adım kurgulamak, API key adimini baslatmak
2	Iteratif yonlendirme, kapsam birlestirme	Docker+Postgres ve Spring kurulumunu tek turda tamamlamak
3	Role prompting, etik ilke enjeksiyonu (FATS), cikti formati dayatma, negatif kisitlar	Etigi API davranisina ceviren endpoint tasarimi
4	Meta-prompting, 6 elementli sablon, gizli CoT, anti-halusinasyon kisiti, JSON sema dayatma, prompt versiyonlama	Gemini uretim promptu + controller implementasyonu
5	Kapili ilerleme onayi	Servis katmani implementasyonuna gecis
6	Hata baglami enjeksiyonu, kapsam sinirlama, gerekcelendirme talebi	503 hatasina karsi retry/backoff ve seffaf hata yonetimi
7	Audience-aware prompting, persona tanimi, negatif kisitlar	Takim arkadası icin minimum adimli kurulum dokumani
8	Iteratif gereksinim rafine etme	Kurulumu Docker-only hale getirmek
9	Oz-referansli ozetleme, durustluk kisiti, format dayatma	Prompt log + etik degerlendirme raporlarini uretmek
10	Role prompting, referans ornekli stil tanimlama, negatif kisit, yapisal ayristirma, cikti formati dayatma	Backend cikti semasiyla eslesen premium frontend arayuzu tasarlamak

SON NOT

Bu oturum boyunca kullanan teknikler, SIC Generative AI modulunde islenen cerceveyle dogrudan eslesiyor:

- Zero-shot: Tur 4'te Gemini icerik uretim promptunun ornek cikti olmadan calistirilan versiyonu.

- Few-shot: Aynı promptun bir örnek çıktı içerecek şekilde zenginleştirilen versiyonu (A/B karşılaştırması için).
- Chain-of-Thought: Tur 4'teki "Düşünce Süreci" bölümü - modelin çıktıya yazmadan adım adım iç muhakeme yapması.
- 6 elementli master şablon (Role/Context/Goal/Constraints/Style/Output Format): Tur 4'te Gemini'ye gönderilen sistem promptunun tamamı.

-e

Etik Değerlendirme

ETİK DEĞERLENDİRME

Eco-Tech Kampanya Motoru - Sorumlu AI Kullanımı Değerlendirmesi

Bu değerlendirme, sistemin tasarımı sırasında fiilen alınan kararlara dayanmaktadır. İlke çerçevesi: Fairness, Accountability, Transparency, Safety (FATS) + Human-in-the-loop.

1. TESPİT EDİLEN RİSKLER VE ALINAN SOMUT ONLEMLER

Risk 1 - Uydurma istatistik ve rakam üretimi (halüsinasyon): Üretim modeli, kampanya metnini ikna edici kılmak için gerçekte var olmayan yüzdeler, tonajlar veya hedef tarihler uydurabilir; bir sürdürülebilirlik kampanyasında bu doğrudan yanıltıcı çevre iddiası (greenwashing) riskidir.

Onlem (iki katmanlı, "modele güvenme, doğrula" ilkesi):

(a) Prompt katmanı: Sistem promptunda, kullanıcı girdisinde veya doğrulanmış veri listesinde yer almayan hiçbir sayısal iddianın üretilmemesi açıkça yasaklandı; rakam retorik olarak gereklirse modelin [KAYNAK DOĞRULANMALI] etiketi yazması zorunlu kılındı. (b) Backend katmanı: Üretilen metinler, modelden bağımsız olarak sayısal iddia kalıplarına (yüzde, ton, milyon vb.) karşı regex ile taranır; kullanıcı girdisinde bulunmayan sayısal iddia tespit edilirse kayda CONTAINS_UNVERIFIED_STATISTIC uyarısı eklenir ve bu uyarı onaylayacak insana API yanıtında gösterilir.

Risk 2 - AI çıktısının insan onayı olmadan yayınlanması: Üretilen içeriğin doğrudan yayına gitmesi, hatalı veya uygunsuz içeriğin marka adına yayılmasına yol açabilir. Onlem: Human-in-the-loop durum makinesi. Her içerik DRAFT durumunda dogar; yalnızca ayrı bir onay endpoint'i (PATCH /status) üzerinden insan kararıyla APPROVED veya REJECTED olabilir. Gecis kuralları sunucu tarafında zorlanır (DRAFT disından gecis 409 ile reddedilir); onay kararı, notu ve zamanı kalıcı olarak kaydedilir.

Risk 3 - Şeffaflık eksikliği (içeriğin AI ürünü olduğunun görünmemesi): İçeriği gören kişi, metnin bir insan tarafından mı yoksa AI tarafından mı üretildiğini ve hangi koşullarda üretildiğini bilemeyebilir. Onlem: Her API yanıtında aiMetadata blogu doner: generatedBy="AI", sağlayıcı, model adı, temperature ve prompt şablon sürümü. Model adı konfigürasyondan okunur; model değişirse kayıtlar doğru kalır. Hata durumları da aynı şeffaflık ilkesiyle tutarlı formatta (timestamp, status, hata kodu, mesaj) raporlanır.

Risk 4 - Hesap verebilirlik eksikliği (üretimin izlenememesi): Hangi çıktının hangi girdi, hangi prompt ve hangi parametrelerle üretildiği kaybolursa sorumluluk atfedilemez. Onlem: Audit trail doğrudan veri modeline gömüldü: her kayıt, kullanıcı girdisini, Gemini'ye giden promptun TAM metnini, prompt sürümünü, model parametrelerini ve zaman damgasını birlikte tasir. Basarisiz üretimler de (JSON doğrulamasından gecemeyen çıktılar ve sağlayıcıya ulaşamayan denemeler) otomatik RED notuyla kalıcı olarak kaydedilir; teknik olarak bu, kayıt işleminin hata fırlatan akıştan etkilenmemesi için üretim metodunun transaction kapsamı dışında tutulmasıyla garanti edildi.

Risk 5 - Hedef kitle kliseleştirilmesi (bias): Prompt, kitle segmentasyonu gereği "Z Kusagı = emojili ve dinamik, kurumsal = ciddi" gibi genellemeler içerir; bu, kitlelerin homojen olduğu varsayımını yeniden üretebilir ve bazı alt grupları dışlayabilir. Onlem: Bu risk otomatik olarak çözülmedi; ana savunma hattımız, hiçbir içeriğin insan değerlendirmesinden gecmeden yayınlanamaması (Risk 2'deki mekanizma). Klise veya dışlayıcı ton, onay aşamasında insan tarafından reddedilebilir ve reviewNote alanına gerekçesi yazılabilir.

Risk 6 - Kötü niyetli veya asiri girdi (prompt injection yüzeyi): Kullanıcı girdisi doğrudan prompta gömüldüğü için, uzun veya manipülatif girdiler modelin talimatlarını ezmeye çalışabilir. Onlem (kısmi): Girdi 10-500 karakterle sınırlandı; bu hem anlamsız girdileri eler hem injection yüzeyini küçültür. Çıktı tarafında kati JSON sema doğrulaması vardır; semaya uymayan her yanıt reddedilir.

2. BİLİNEN SINIRLILIKLAR

- Kim onayladı bilgisi tutulmuyor: Kimlik doğrulama (auth) hackathon

kapsami disinda birakildigi icin onay kararinin KIM tarafından verildigi kaydedilemiyor; yalnızca karar, not ve zaman tutuluyor. Gerçek bir urunde onaylayan kullanıcı kimligi audit kaydinin zorunlu parçasi olmalıdır.

- İstatistik taraması kabadir: Regex tabanlı tarama yazılı sayıları ("bes yuz ton" gibi) veya dolaylı iddiaları kacirabilir, masum sayıları da yanlış isaretleyebilir. Uyarı mekanizması engelleyici değil, insana sinyal verici olarak tasarlandı; nihai karar insanda.
- Modelin kisitlara uyma garantisi yoktur: [KAYNAK DOGRULANMALI] etiketi ve sayisal iddia yasagi prompt duzeyinde talimattir; model bunlara uymayabilir. Backend taraması bu nedenle vardır ama o da eksiksiz değildir (bkz. önceki madde).
- Bias için otomatik tespit yok: Kitle klisesi, ton uygunsuzluğu veya displayici dil için otomatik bir sınıflandırıcı kullanılmıyor; tamamen insan denetimine dayanılıyor.
- Doğrulanmış veri listesi şu an boş: Prompt, doğrulanmış Samsung/UNDP verilerini kullanacak şekilde tasarlandı ancak bu listeye henüz gerçek veri girilmedi; girilene kadar mesru sayisal iddialar da üretilemez (güvenli taraf tercih edildi).
- İçerik güvenliği taraması kapsam dışı: Toksisite, nefret söylemi gibi kategoriler için ayrı bir filtre eklenmedi; bu da insan onayı adına bırakıldı.

Sonuç: Sistem, etik ilkeleri prompt süslemesi olarak değil, API davranışı olarak uygular - durum makinesi, audit alanları ve uyarı mekanizması kodda doğrulanabilir durumdadır. Çözülmemiş riskler bilinçli olarak insan denetimi katmanına devredilmiş ve burada açıkça belgelenmiştir.